Programación Entera y Branch and Bound

De la relajación LP a la poda — ejemplos enteros y mixtos, y dos formas de resolverlo en Python

---

Métodos de Optimización — Maestría

Universidad Pontificia Bolivariana

Motivación: de PL continua a PL entera

¿Dónde estamos?

En la Clase 4 resolvimos programación lineal (PL): variables **continuas**, simplex exacto en un número finito de pasos.

Pero muchos problemas reales exigen variables **enteras**: número de mesas a fabricar, camiones a despachar, plantas a construir, o decisiones sí/no (0 o 1).

El problema

El vértice óptimo que encuentra el simplex **casi nunca** cae exactamente en un punto de coordenadas enteras. No podemos simplemente "redondear" el resultado: redondear puede violar restricciones, o alejarnos bastante del verdadero óptimo entero.

Hoy: **Branch and Bound** (ramificación y acotamiento) — el método estándar para resolver PL entera y entera mixta, construido enteramente sobre el simplex que ya dominamos.

Programación entera: tres sabores

Tipo	Qué exige
PL continua (Clase 4)	Todas las variables $\in \mathbb{R}$, ≥ 0.
PL entera (pura)	**Todas** las variables $\in \mathbb{Z}$, ≥ 0.
PL entera **mixta** (MIP)	**Algunas** variables $\in \mathbb{Z}$, el resto $\in \mathbb{R}$.

Hoy veremos: un ejemplo entero puro sin poda "real", un ejemplo entero puro **con** poda genuina, y un ejemplo mixto. Los tres, verificados numéricamente y resueltos con dos librerías de Python distintas al final.

La idea central: relajación, ramificación y poda

Paso 1: la relajación LP

Relajación (*LP relaxation*)

Toma el problema entero y **borra temporalmente** la exigencia de integralidad. Lo que queda es un PL continuo ordinario: lo resuelves con el simplex de la Clase 4, sin ningún cambio.

- La región factible del problema entero es un **subconjunto** (los puntos de malla dentro) de la región factible del LP relajado.
- Por lo tanto, el óptimo del LP relajado es siempre **igual o mejor** que el óptimo entero real — nunca peor.
- Ese valor es una **cota**: en maximización, una **cota superior** que ningún entero factible puede superar.

Si la solución del LP relajado ya sale con todas las variables enteras (declaradas como tales): **listo, es también el óptimo entero** — no hace falta hacer nada más.

Paso 2: ramificar

Si alguna variable entera sale fraccionaria (digamos $x_j = 3{,}4$), ni $x_j \leq 3$ ni $x_j \geq 4$ excluyen ningún entero factible — pero juntas cubren *todo* lo que antes cubría x_j sin restricción.

Ramificación (*branching*)

Se crean **dos subproblemas hijos**, idénticos al padre más una restricción adicional:

Hijo izquierdo: $x_j \leq \lfloor 3{,}4 \rfloor = 3$ Hijo derecho: $x_j \geq \lceil 3{,}4 \rceil = 4$

Cada hijo es, de nuevo, un simple problema de PL continua: se resuelve con simplex, exactamente igual que el padre. Si vuelve a salir fraccionario, se ramifica otra vez — así se construye un **árbol** de subproblemas.

Paso 3: por qué esto no es fuerza bruta — la poda

Sin ningún control, el árbol de ramificación puede crecer exponencialmente (piense en cuántas variables fraccionarias pueden aparecer, en cascada). Lo que lo hace práctico es la **poda** (*pruning* / *fathoming*).

La idea de la poda, en una frase

Como la relajación de cualquier nodo del árbol es una **cota** sobre lo mejor que se puede lograr bajando por esa rama, en cuanto esa cota deja de superar a la mejor solución entera ya encontrada, **se descarta la rama completa sin explorarla más**.

El resto de esta clase dedica una sección entera a precisar exactamente cuándo se poda y cuándo no — es el punto donde más fácil se confunden los estudiantes.

El concepto de poda, a fondo

El “incumbente”

Definición

El **incumbente** es la mejor solución **entera factible** encontrada hasta el momento durante la exploración del árbol. Se actualiza cada vez que aparece una solución entera mejor; nunca empeora.

Al principio (antes de encontrar cualquier entero) no hay incumbente: cualquier entero factible que aparezca se vuelve el primero. A partir de ahí, cada nodo nuevo se compara contra él.

El incumbente cumple dos papeles: (1) es el candidato a **respuesta final** si no aparece nada mejor, y (2) es la **vara de medir** contra la que se prueba cada cota para decidir si podar.

Tres razones distintas para terminar una rama

Razón	Qué pasa
Nodo **infactible**	El LP del nodo no tiene solución (las restricciones acumuladas se contradicen). No hay nada ahí: se descarta.
Nodo da solución **entera**	Es una hoja natural: ya no queda variable entera fraccionaria. Se compara contra el incumbente; si es mejor, lo reemplaza. **No es "poda" en sentido estricto** — la rama simplemente ya terminó por sí sola.
Cota del nodo **no supera** al incumbente	**Poda genuina** (*fathoming by bound*): aunque la solución sea fraccionaria y en principio podría ramificarse más, se descarta **sin** explorar sus hijos, porque ningún descendiente puede superar al incumbente.

El matiz que más se confunde

Ojo con esto

Encontrar una solución entera *peor* que el incumbente y descartarla **no es poda genuina** — es solo la comparación final de una hoja que de todos modos ya no tenía a dónde seguir. **La poda de verdad ocurre *antes* de llegar a un entero**: en un nodo todavía fraccionario, cuando su cota ya pierde.

No es poda genuina	**Sí es poda genuina**
Nodo da un entero directamente, y ese entero resulta peor que el incumbente.	Nodo sigue fraccionario (tendría hijos por explorar), pero su cota ya es $\leq$ el incumbente: se descartan esos hijos **sin calcularlos**.

Este matiz distingue nuestro Ejemplo 1 (ninguna poda genuina) del Ejemplo 2 (sí la hay) — vamos a verlo con números.

Ejemplo 1: sin poda genuina

Ejemplo 1: planteamiento

Problema

$$\text{máx } z = 5x_1 + 4x_2 \quad \text{s.a.} \quad 6x_1 + 4x_2 \leq 24, \quad x_1 + 2x_2 \leq 6, \quad x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}$$

Relajación LP (resuelta con simplex, Clase 4):

$$x_1^{LP} = 3, \quad x_2^{LP} = 1{,}5, \quad z^{LP} = 21$$

$x_2 = 1{,}5$ es fraccionario $\rightarrow$ no es solución entera válida, pero $z^{LP} = 21$ ya nos da la **cota superior global**: ningún entero factible puede superar 21.

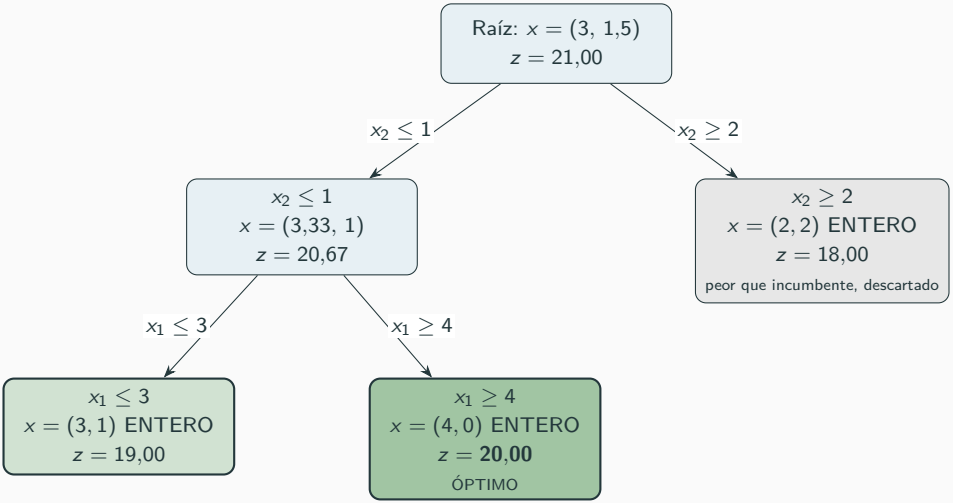
Ramificamos sobre x_2: hijo izquierdo $x_2 \leq 1$, hijo derecho $x_2 \geq 2$.

Ejemplo 1: resolviendo cada nodo

Nodo	**Restricción**	**Solución LP**	z
Raíz	—	$(3,\ 1{,}5)$	21,00
2	$x_2 \leq 1$	$(3{,}33,\ 1)$	20,67
3	$x_2 \geq 2$	$(2,\ 2)$ — **entero**	18,00
4	$x_2 \leq 1,\ x_1 \leq 3$	$(3,\ 1)$ — **entero**	19,00
5	$x_2 \leq 1,\ x_1 \geq 4$	$(4,\ 0)$ — **entero**	**20,00**

Orden de exploración: primero se agota la rama $x_2 \leq 1$ (nodos 4 y 5) antes de visitar el nodo 3. Al llegar al nodo 3, el incumbente ya es 20 (del nodo 5) — el entero del nodo 3 (18) queda descartado por simple comparación.

Ejemplo 1: árbol completo



Ejemplo 1: por qué NO hubo poda genuina

Revisando el matiz

El único nodo "descartado" (nodo 3, $x_2 \geq 2$) llegó **directo a una solución entera** ($z = 18$). No teníamos ninguna cota fraccionaria que descartar de antemano — simplemente comparamos dos hojas y nos quedamos con la mejor. **Se calcularon los 5 nodos completos**, sin ahorrarnos ningún trabajo.

Óptimo entero: $x_1^* = 4,\ x_2^* = 0,\ z^* = 20.$

Con esto en mente, veamos un ejemplo donde sí ocurre una poda genuina — donde una rama fraccionaria se descarta *sin* calcular sus hijos.

Ejemplo 2: con poda genuina

Ejemplo 2: planteamiento

Problema

$$\text{máx } z = 5x_1 + 3x_2 \quad \text{s.a.} \quad 2x_1 + 3x_2 \leq 12, \quad 3x_1 + x_2 \leq 9, \quad x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}$$

Relajación LP:

$$x_1^{LP} = 2{,}1429, \quad x_2^{LP} = 2{,}5714, \quad z^{LP} = 18{,}4286$$

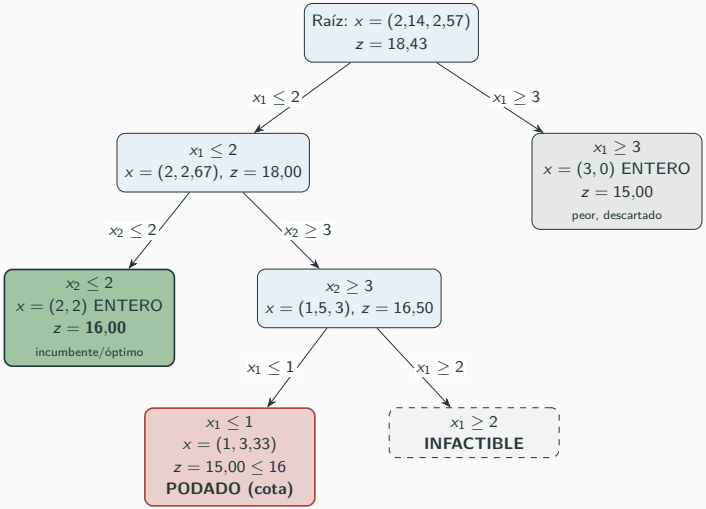
Ambas variables fraccionarias: se ramifica sobre una (típicamente la más fraccionaria, o simplemente la primera fraccionaria en orden — aquí x_1).

Ejemplo 2: resolviendo el árbol

Nodo	Restricción acumulada	Solución LP	z
Raíz	—	(2,14, 2,57)	18,43
2	$x_1 \leq 2$	(2, 2,67)	18,00
3	$x_1 \leq 2,\ x_2 \leq 2$	(2, 2) — **entero**	16,00
4	$x_1 \leq 2,\ x_2 \geq 3$	(1,5, 3)	16,50
5	$\ldots,\ x_1 \leq 1$	(1, 3,33)	**15,00**
6	$\ldots,\ x_1 \geq 2$	**infactible**	—
7	$x_1 \geq 3$	(3, 0) — **entero**	15,00

Tras el nodo 3, el incumbente es 16. El nodo 5 da $x = (1,\ 3{,}33)$ — x_2 sigue fraccionario, en cualquier otro caso tocaría ramificar de nuevo. Pero $z = 15 \leq 16$: **se poda sin calcular los hijos de este nodo**.

Ejemplo 2: árbol completo



Ejemplo 2: cuantificando el ahorro

Lo que la poda nos ahorró

El nodo 5 tenía $x_2 = 3{,}33$ fraccionario. Sin poda, habría que ramificar sobre x_2: crear un hijo $x_2 \leq 3$ y otro $x_2 \geq 4$, resolver ambos LP, y potencialmente seguir ramificando si alguno vuelve a salir fraccionario. **Ninguno de esos cálculos se hizo** — se descartaron de un solo golpe porque $15 \leq 16$ hacía imposible que mejoraran el incumbente.

Óptimo entero: $x_1^* = 2,\ x_2^* = 2,\ z^* = 16.$

En árboles pequeños como este el ahorro parece modesto (2 nodos hijos menos). En problemas reales, con decenas de variables enteras, la poda es lo que hace la diferencia entre un problema que se resuelve en segundos y uno que nunca termina.

Ejemplo 3: entero mixto

Ejemplo 3: la única regla que cambia

Recordatorio

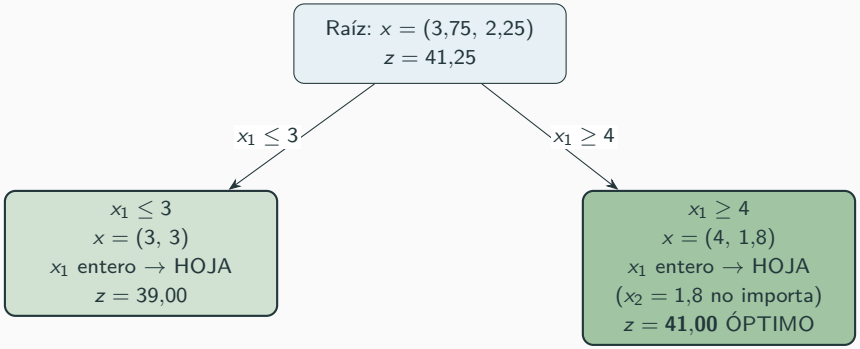
En PL entera **mixta**, solo se revisa fraccionalidad — y solo se ramifica — sobre las variables **declaradas enteras**. Las variables continuas pueden quedar con cualquier valor decimal: nunca generan ramificación, y no impiden que un nodo sea una hoja válida.

Problema

$$\text{máx } z = 8x_1 + 5x_2 \quad \text{s.a.} \quad x_1 + x_2 \leq 6, \;\; 9x_1 + 5x_2 \leq 45, \;\; x_1, x_2 \geq 0, \;\; \mathbf{x_1} \in \mathbb{Z}, \; x_2 \in \mathbb{R}$$

Relajación LP: $x_1^{LP} = 3{,}75$, $x_2^{LP} = 2{,}25$, $z^{LP} = 41{,}25$. Solo miramos $x_1 = 3{,}75$: fraccionario $\rightarrow$ ramificar sobre x_1 únicamente.

Ejemplo 3: árbol (solo 2 niveles)



Ambos hijos ya son hojas válidas apenas x_1 sale entero — x_2 nunca se revisa ni se ramifica, aunque sea decimal en los dos casos.

Ejemplo 3: comparación con el caso entero puro

	Entera pura	**Entera mixta**
¿Qué se revisa?	Todas	Solo enteras declaradas
¿Sobre qué se ramifica?	Cualquier fraccionaria	Solo enteras fraccionarias
¿Cuándo es hoja?	Todas enteras	Declaradas enteras ya enteras
Poda, cota, incumbente	Igual	Igual

Óptimo: $x_1^* = 4,\ x_2^* = 1{,}8,\ z^* = 41.$

La idea para llevar

B&B no distingue entre PL entera pura y mixta a nivel de algoritmo: la única diferencia vive en la pregunta "¿esta solución ya es válida?" hecha en cada hoja.

Verificación computacional

Código 1: `scipy.optimize.milp`

`scipy` incluye un solver de PL entera mixta desde la versión 1.9, construido sobre HiGHS (branch-and-bound + planos de corte en C++):

```
from scipy.optimize import milp, LinearConstraint, Bounds

res = milp(
    c=[-5, -4], # milp SIEMPRE minimiza
    constraints=LinearConstraint([[6,4],[1,2]], -float("inf"), [24,6]),
    integrality=[1, 1], # 1=entera, 0=continua; por variable
    bounds=Bounds(0, float("inf")),
)
print(res.x, -res.fun) # -> [4. 0.] 20.0
```

`integrality` es un vector: `1` marca cada variable que debe ser entera, `0` la deja continua — así se resuelven, con la misma función, tanto el caso puro como el mixto (Ejemplo 3 solo cambia a `integrality=[1,0]`).

Resultados de `milp` en los 3 ejemplos

	Ejemplo 1	**Ejemplo 2**	**Ejemplo 3 (mixto)**
`integrality`	`[1,1]`	`[1,1]`	`[1,0]`
x^*	$(4,\ 0)$	$(2,\ 2)$	$(4,\ 1{,}8)$
z^*	20	16	41

Los tres coinciden exactamente con lo obtenido a mano, nodo por nodo, en las secciones anteriores.

Código 2: PuLP (alternativa recomendada)

PuLP escribe el modelo de forma **declarativa** — casi como en el papel, sin armar matrices A, b, c a mano. Por dentro llama al solver libre CBC (COIN-OR Branch and Cut), que hace el mismo branch and bound que vimos (más refinamientos como planos de corte):

```
import pulp

prob = pulp.LpProblem("mip", pulp.LpMaximize)
x1 = pulp.LpVariable("x1", lowBound=0, cat="Integer")
x2 = pulp.LpVariable("x2", lowBound=0, cat="Integer")

prob += 5*x1 + 4*x2 # funcion objetivo
prob += 6*x1 + 4*x2 <= 24
prob += x1 + 2*x2 <= 6

prob.solve(pulp.PULP_CBC_CMD(msg=0))
print(x1.value(), x2.value(), pulp.value(prob.objective)) # 4.0 0.0 20.0
```


Instalación: `pip install pulp` (CBC viene empaquetado, no se instala aparte).

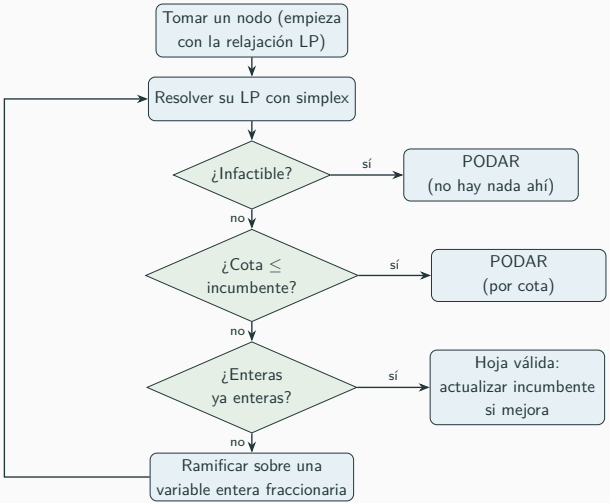
`scipy.optimize.milp` vs. PuLP: ¿cuándo cuál?

	`scipy.optimize.milp`	**PuLP + CBC**
Sintaxis	Matricial (A, b, c, como en la Clase 4)	Declarativa ("como en el papel")
Solver interno	HiGHS (C++, el mismo de `linprog`)	CBC (COIN-OR), intercambiable
Cuándo conviene	Ya se tiene el modelo matricial, o se usa solo scipy	Modelos grandes/legibles, buena para enseñar

Otras alternativas del mismo estilo declarativo: `OR-Tools` (Google, CP-SAT muy fuerte en variables binarias), `Pyomo` (más general, cambia de solver fácilmente), y los solvers comerciales `Gurobi` / `CPLEX` (mucho más rápidos en problemas grandes, con licencia académica gratuita).

Síntesis y cierre

Mapa conceptual de Branch and Bound



Fuerza bruta vs. Branch and Bound

Recordando la Clase 4: fuerza bruta en PL continua enumeraba $\binom{n}{m}$ vértices. En PL entera, el equivalente de fuerza bruta sería enumerar **todos** los puntos enteros dentro de la región factible.

Por qué B&B gana

B&B nunca calcula un nodo cuya cota ya pierde contra el incumbente. Cada poda elimina, de un solo golpe, **todo un subárbol** que fuerza bruta habría tenido que recorrer entero. En problemas con decenas o cientos de variables enteras, esta diferencia es la que separa un problema resoluble de uno computacionalmente imposible.

Para tus estudiantes, en una frase: B&B no es un algoritmo nuevo — es el simplex de la Clase 4, orquestado en un árbol con una regla de poda encima.

Lo que ya sabemos hacer

- Construir la relajación LP de un problema entero y leerla como cota.
- Ramificar sobre una variable fraccionaria sin perder soluciones enteras.
- Distinguir con precisión una poda genuina (por cota, antes de terminar la rama) de una simple comparación entre hojas.
- Resolver un árbol de B&B completo a mano, para problemas enteros puros y mixtos.
- Verificar cualquier resultado con `scipy.optimize.milp` o con `PuLP`.

Lo que falta (próxima clase / roadmap)

- **Variables binarias** $(0/1)$: el caso especial más común en la práctica (decisiones sí/no) — mismo B&B, ramificación siempre en $x_j = 0$ vs. $x_j = 1$.
- **Planos de corte** (*cutting planes*): en vez de solo ramificar, añadir automáticamente restricciones válidas que "cortan" la solución fraccionaria sin eliminar ningún entero — lo que usan CBC y HiGHS por dentro, combinado con B&B (*branch and cut*).
- **Estrategias de selección**: qué variable fraccionaria ramificar primero, y en qué orden explorar el árbol (profundidad vs. mejor cota primero) — afecta mucho el tiempo de cómputo en problemas grandes.
- Esto conecta con el roadmap general del curso: tras PL y PL entera, el siguiente territorio natural es la programación no lineal con restricciones (SQP, barreras) ya anticipada en la Clase 4.

Ejercicios propuestos

1. Resuelva a mano el árbol de B&B para máx $z = 3x_1 + 2x_2$ s.a. $x_1 + x_2 \leq 4,\ x_1 \leq 3,\ x_2 \leq 3,\ x_1, x_2 \in \mathbb{Z}$. ¿Hay poda genuina?
2. Para el Ejemplo 2 de la clase, en vez de ramificar primero sobre x_1, intente ramificar primero sobre x_2 en la raíz. ¿Llega al mismo óptimo? ¿El árbol tiene el mismo número de nodos?
3. Modifique el Ejemplo 3 declarando también x_2 como entera. Resuelva el nuevo árbol y compare cuántos nodos hacen falta frente a la versión mixta.
4. Investigue qué es una variable binaria y reescriba el Ejemplo 1 usando una variable $y \in \{0, 1\}$ que module si se usa o no la restricción $x_1 \leq 4$ (pista: técnica de "Big M").
5. Verifique los tres ejemplos de la clase con `OR-Tools` (CP-SAT) y compare el número de nodos explorados que reporta contra los que calculamos a mano.
6. ¿Qué pasaría si, en el Ejemplo 2, el incumbente inicial fuera 17 en vez de 16 al llegar al nodo 5? Verifique si la poda seguiría aplicando.

Referencias

- Wolsey, L. A. (2020). *Integer Programming* (2nd ed.). Wiley.
- Nemhauser, G. L., & Wolsey, L. A. (1988). *Integer and Combinatorial Optimization*. Wiley.
- Land, A. H., & Doig, A. G. (1960). "An automatic method of solving discrete programming problems". *Econometrica*, 28(3), 497–520. (Origen histórico de Branch and Bound.)
- Bertsimas, D., & Weismantel, R. (2005). *Optimization over Integers*. Dynamic Ideas.
- Mitchell, S., O'Sullivan, M., & Dunning, I. (2011). *PuLP: A Linear Programming Toolkit for Python*. Documentación oficial: `https://coin-or.github.io/pulp/`.
- Virtanen, P., et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python". *Nature Methods*, 17, 261–272.
- Forrest, J., & Lougee-Heimer, R. (2005). *CBC User Guide*. COIN-OR Foundation.
- Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.), Cap. 13. Springer.

Gracias

Preguntas y discusión